

Brody Validation Report

Full validation of the Brody repository: syntax, correctness, performance, styling and accessibility, live AI behaviour and cost, delivery formats and deployment.

Run on 2026-09-23 against the working tree, macOS, Node 25, Chrome, Docker Desktop, Lean 4.34.0, with OpenAI gpt-5 for the live AI runs.

Results come from commands that were actually run; nothing here is estimated. The previous validation (2026-09-20) predates formal verification with Lean, the AI usage and cost gauge, and the parse worker threads.

Contents

1. Verdict	2
2. Method	2
3. Syntax and static validation	2
4. Functional validation	3
5. Self-analysis: Brody analysing Brody	3
6. Live AI validation (OpenAI gpt-5)	4
7. Performance	5
8. Styling and accessibility	6
9. Viewing and downloading results	6
10. Deployment	7
11. Not verified and known limitations	7

To repeat every automated check: `npm run validate` (types, lint, tests, both production builds, performance budgets, browser suite). CI runs the same command.

1. Verdict

Area	Result	Evidence
Syntax and types	Pass	tsc --noEmit clean; ESLint 0 errors and 0 warnings; Turbopack and webpack production builds with 0 warnings (after fixes)
Functional correctness	Pass	379 unit and integration tests, all passing
Browser end to end	Pass	40 Playwright tests in Chrome, all passing
Performance	Pass, with a recorded trade-off	3,000 files analyse in 6.1 s (was 11.6 s); parse workers use more memory; budgets now enforced
Styling and layout	Pass after fixes	4 widths, 5 brightness levels, 13 views plus a populated Formal Proofs page; 2 layout defects fixed
Accessibility	Pass	axe WCAG 2 A and AA: 0 serious or critical violations on every view at every brightness level
Self-analysis	Pass after fixes	38 findings became 25 (0 Critical, 0 High); 5 analyzer imprecisions and 2 parser false alarms fixed at the source
Live AI (OpenAI gpt-5)	Pass after fixes	2 defects found and fixed: suggested patches were all discarded, and Lean repair rounds were wasted
AI cost transparency	Pass	The usage gauge recorded every live request, with model and cost
Downloads	Pass after a fix	Reports labelled Lean-proved findings "AI-inferred"; fixed
Docker	Pass	Image builds with Lean included, healthy in 8 s, non-root, analysis and exports work inside the container

2. Method

1. Static checks: TypeScript compiler, ESLint with zero warnings allowed, both Next.js bundlers with their warnings treated as failures.
2. Automated tests: Vitest for logic, API handlers, security, exports, formal verification (against the real Lean kernel) and the analysis pipeline; Playwright for the real UI.
3. Self-analysis: Brody analysed its own repository with its own pipeline. Every finding was read. Analyzer mistakes were fixed in the analyzer with a regression test; genuine issues were fixed or recorded with a reason.
4. Live AI: two full analyses of the sample repository against OpenAI gpt-5 (`npm run verify:ai`), before and after fixes, plus one small request to capture the model's patch format.
5. Performance: the synthetic benchmark at 200, 1,000 and 3,000 files, a memory profile of the self-analysis, and web timing budgets in Playwright.
6. Styling: screenshots of every view at 390, 820, 1,100 and 1,440 px in the default and darkest levels, reviewed by eye as contact sheets, plus automated overflow, console-error and axe checks at all five levels.
7. Deployment: Docker image build and run with a real upload, analysis and export, and Lean checked as the container user.

3. Syntax and static validation

Check	Result
tsc --noEmit	No errors, including tests, e2e specs and scripts
ESLint	0 errors, 0 warnings in 206 files

Check	Result
next build (Turbopack)	Compiles; 2 warnings fixed (see below), now 0
next build --webpack (Docker)	Compiles; 1 warning fixed, now 0
Brody's TypeScript syntax check on Brody	0 syntax errors
Brody's ESLint adapter on Brody	0 findings after the fix below

Defects found and fixed:

Defect	Effect	Fix
The AI settings file was read through a path computed at run time	Turbopack traced the whole project into the server output	The reads and writes are marked as runtime data, not code
The parse pool found <code>tsx</code> with a module lookup	Webpack tried to bundle <code>tsx</code> , which broke the Docker build since the parse-worker commit; the first fix only turned that into a warning	<code>tsx</code> is found by checking for its <code>package.json</code> on disk
The rule engine loop could, in principle, never advance on a pattern matching an empty string	A possible hang on a future rule (ESLint pointed at the loop)	The loop always advances and checks its limit explicitly
The Node 25 message <code>--localstorage-file</code> was provided without a valid path	Printed by Next.js's own build workers on Node 25	Not Brody's flag; not printed on Node 24, which the Docker image and CI use. Recorded, not changed

4. Functional validation

379 tests across 27 files, all passing. New since the previous report:

Suite	What it proves
Formal verification (16)	The Lean source policy refuses code execution and fake proofs; theorems are proved, counterexamples are proved, failures are reported with Lean's error; a printed fake axiom report is ignored; theorems written into a model are removed; true claims are confirmed and false ones refuted end to end; repairs stop when they stall; unchanged functions reuse their proofs; an unfaithful model changes nothing
AI usage and cost (9)	Prices by model and platform prefix, overrides, cost arithmetic, per-model and per-step totals, isolation between concurrent runs, live usage saved during a run
Helpers (7)	The text and client helpers most of the code relies on
Precision (5 new)	Multi-line template strings are not reviewed as code; an explained empty catch is deliberate; retry loops and variable hosts are not flagged; an empty regex match cannot hang the engine
Patches (1 new)	Hunks without line numbers apply only where they match exactly one place
Embeddings (1 new)	Parallel batches keep every vector in input order

The worker-thread tests also caught a stale compiled worker (`dist/parse-worker.cjs`) during this validation. The test suite now rebuilds it before it runs, so a stale bundle can never hide a source change again.

Browser tests (40) cover the whole workflow, the downloads, four widths at two brightness levels, accessibility at all five levels, keyboard use, performance budgets, the AI usage gauge (live updates, popover, phone layout) and the Formal Proofs page populated with a realistic Lean report at three widths and five levels.

5. Self-analysis: Brody analysing Brody

Clean run, deterministic (no AI): 259 files, 48,377 lines of code, 2,066 symbols, 5,582 relationships, 44 routes, 12 data models, 29 functional areas, in 10.1 seconds. The pipeline peaks at 860 MB; the command-line tool reaches 1.14 GB because it then writes every report format at once. The reports are in [self-analysis/](#).

The first run reported 38 findings and 2 parse warnings. Each class of mistake was fixed at the source, with a test:

Problem in the analyzer	Effect on Brody	Fix
Lines in the middle of a multi-line template string were treated as code	3 empty-catch findings inside the browser scripts Brody embeds as strings (the deck presenter and the theme script)	The literal blanker carries template state across lines
The empty-catch rule matched text inside strings	Same	The rule matches code only; a catch holding an explanatory comment still counts as deliberate
A retry loop counted as "sequential await in a loop"	The OpenAI client's retry loop was flagged	Loops over attempt/retry counters are exempt; retries are sequential by design
A URL whose host is a variable counted as a hard-coded plain HTTP call	The local launcher's <code>http://\$HOST:\$PORT</code> was flagged	Variable hosts are exempt, like <code>\${...}</code> already was
The tree-sitter grammar's gaps were reported as syntax errors	2 "syntax errors present" warnings on valid files (a bare <code>&</code> in a JSX string, <code>unique</code> used as a variable name)	When the TypeScript compiler accepts a file, a grammar-only error is not reported

Genuine findings fixed in Brody: the OpenAI embedding requests ran one batch at a time (now 4 in parallel, order preserved); two small files were re-read on every call (now read once); the three most-depended-on files without tests now have them. Deliberate constructs (the theme script injected before first paint, a lazily created linter, tiny settings reads) carry a `brody-ignore` with the reason.

Result: 38 findings became 25 (0 Critical, 0 High, 8 Medium, 17 Low). No finding comes from the pattern rules or ESLint any more. Remaining, all accepted:

Finding	Assessment
8 Medium: very high cyclomatic complexity in <code>detectRoutes</code> (190), <code>extractTsJs</code> (166), <code>buildGraph</code> (156), <code>buildDeckContent</code> (123), <code>Home</code> (110), <code>buildMarkdown</code> (109), <code>enhanceWithAI</code> (107), <code>detectEntryPoints</code> (100)	Real technical debt: long tables of framework rules, and large pipeline and UI functions. The tests around them are strong, so splitting them is a safe future task, and a separate piece of work
8 Low: long functions and one long component in the same places	Same cause
1 Low: none of the 33 API routes has an authentication check	True and documented: Brody is single-tenant and belongs behind an authenticating proxy
7 Low: secret-like values in tests and the fixture	Intentional fake credentials used to test secret detection
1 Low: 3 important files without direct tests	Two now have direct tests; the third is React components, covered by the browser tests. The rule counts only direct imports from test files

6. Live AI validation (OpenAI gpt-5)

`npm run verify:ai` analyses the 17-file sample shop with every AI stage on. Numbers come from Brody's own usage gauge, which recorded every request.

	Run 1 (before fixes)	Run 2 (after fixes)
Result	All 14 stages done; 1 request timed out	All 14 stages done; no request failed
Wall time	40 min	36 min
Requests	62	61
Tokens (input / output)	112k / 311k	94k / 319k
Estimated cost	\$3.25	\$3.32
of which formal verification (Lean)	33 requests, \$1.84 (57%)	32 requests, \$2.04 (61%)
Findings	35 (29 verified, 6 need verification)	34 (29 verified, 5 need verification)

	Run 1 (before fixes)	Run 2 (after fixes)
Suggested patches kept	0: every one discarded	18 kept; 4 discarded because they do not match the file
Lean	9 of 10 functions modelled, 33 theorems proved, 5 AI claims confirmed with a concrete input (SQL injection, insecure direct object reference, swallowed payment errors), 17 proofs set aside by the fidelity audit	7 of 7 functions modelled (the eval endpoint is no longer sent), 26 theorems proved, 5 AI claims confirmed with a concrete input, 0 unproven (was 5), 13 proofs set aside by the audit (was 17)

Defects found by the live runs and fixed:

Defect	Cause	Fix
Every suggested patch from gpt-5 was discarded ("no hunks found")	gpt-5 writes hunk headers without line numbers (@@), captured with a single probe request; the patch checker required @@ -12,3 +12,4 @@	Header-less hunks are accepted and located by their content; a hunk that matches more than one place is still refused, so a patch never lands in the wrong place
Lean repair rounds were wasted	The model also wrote its theorems inside its model, so Lean rejected every repaired file ("has already been declared")	Theorems are removed from model text before checking
Repairs continued when they made no progress	Rounds ran to the limit regardless	A round that proves nothing new ends the repairs
Claims about eval were modelled in Lean	A model of a stand-in evaluator proves nothing about eval; the audit set all five aside, after they were paid for	Claims about dynamic code execution are not sent to Lean

The fixes made the AI stages reliable and useful (no failed request, every patch that fits kept, nothing left unproven) but did not make them cheaper: each Lean request to a reasoning model like gpt-5 spends several thousand output tokens, and formal verification remains about 60% of the cost of a run. The levers are FORMAL_MAX_TARGETS (10 by default), FORMAL_MAX_REPAIR_ROUNDS and FORMAL_VERIFICATION=off; the usage gauge shows the effect of each while a run is going.

The live runs also confirmed the design choices that matter most: the fidelity audit rejected every proof whose model did not match the code, and no proof reached the review without it.

7. Performance

7.1 Analysis pipeline

Synthetic TypeScript repositories, no AI. Times in milliseconds.

Source files	Lines	Parse	Whole pipeline	Previous pipeline	Peak memory	Previous memory
200	8,169	1,265	1,691	1,263	492 MB	491 MB
1,000	41,228	2,134	3,351	3,737	1,604 MB	620 MB
3,000	125,316	2,402	6,065	11,565	2,085 MB	971 MB

From 400 files up, parsing runs on up to four worker threads, so a 3,000-file repository now analyses in about half the time. The cost is memory: each worker holds its own grammars, TypeScript and ESLint (about 300 MB). The worker count is chosen from the memory available, so a 2 GB container runs in-process with the previous footprint; PARSE_WORKERS sets it explicitly.

Interactive operations at 3,000 files: change impact 38 ms, warm search 49 ms, Markdown report 144 ms, full PDF 0.5 s (was 1.5 s), Word 0.3 s (was 0.6 s).

New: `npm run benchmark -- --budget` fails when the pipeline, memory, search, change impact or PDF export exceed generous budgets (for example, a 1,000-file pipeline over 9 s or 1.9 GB). `npm run validate` and CI run it,

so a quadratic step, a lost cache or a leak now fails the build.

7.2 Web performance, production build

Route	DOMContentLoaded	Largest paint	JavaScript transferred
Home	50 ms	148 ms	159 KB
Overview	37 ms	236 ms	166 KB
Code Review	35 ms	200 ms	164 KB
System Explanation	35 ms	200 ms	170 KB
Architecture	37 ms	256 ms	232 KB
Code Map	39 ms	236 ms	230 KB
Files	34 ms	240 ms	192 KB
Ask	29 ms	156 ms	164 KB
Reports	34 ms	168 ms	163 KB
Formal Proofs (new)	36 ms	180 ms	164 KB

API endpoints answered in 3 to 13 ms. Budgets enforced by the tests: DOMContentLoaded under 2.5 s, largest paint under 3.5 s, JavaScript under 700 KB, APIs under 800 ms.

8. Styling and accessibility

Views checked: Home, Overview, Code Review, System Explanation (three scales), Architecture, Code Map, Files, Ask, Executive Deck, Reports and, new, Formal Proofs (empty and populated) and the AI usage gauge and popover. Widths 1,440, 1,100, 820 and 390 px; layout at the default and darkest levels, contrast at all five.

Check	Result
Horizontal page overflow	None at any width, after the fix below
Browser console errors	None
axe-core, WCAG 2 A and AA, 2.1 AA	0 serious or critical violations on every view at every brightness level
Keyboard	Search shortcut; menus and the usage popover close with Escape and return focus

Defects found by this testing and fixed:

Defect	Fix
Formal Proofs on a phone: a long file path could not wrap and pushed the page 144 px wider than the screen	Paths and identifiers wrap anywhere
Phone header: the AI usage gauge took a row of its own, then pushed the theme button onto another	The gauge sits with the header buttons; without AI it shows only its icon on phones (its label still reads "No AI used in this run" to screen readers)

9. Viewing and downloading results

Unchanged from the previous report, with one fix: the Markdown, PDF and Word reports labelled findings proved by Lean as "AI-inferred". They now say "Proved (Lean 4)", and the summary counts them separately. A test holds this.

10. Deployment

Check	Result
docker build	Builds with 0 bundler warnings (it failed to build before this validation's webpack fix); image 4.89 GB including Lean
Container	Healthy in 8 s; runs as node with all capabilities dropped
Lean in the image	4.34.0, reduced to what proof checking needs (1.9 GB of the image)
Address	Answers as brody; any other host name refused with 421
Inside the container	A ZIP upload analysed through all 14 stages; full PDF exported

11. Not verified and known limitations

1. **Live AI with Anthropic.** The live runs used OpenAI gpt-5, the configured provider. The Anthropic request path is verified against test doubles only.
2. **AI run time and cost.** With gpt-5, a 17-file project takes about 36 minutes and \$3.32, about 60% of it formal verification. Large repositories cost proportionally more; the usage gauge shows it live, and FORMAL_MAX_TARGETS or FORMAL_VERIFICATION=off bound it.
3. **Memory with worker threads,** as described in 7.1.
4. **Technical debt:** the 8 very complex functions listed in section 5.
5. **Dependencies.** npm reports ESLint 9.39 as no longer supported and two deprecated @esbuild-kit packages pulled in by drizzle-kit. Upgrading ESLint to the next major version, together with eslint-config-next, is a separate task.
6. The interface was verified in Chrome only (desktop, tablet and phone widths).
7. Brody has no user authentication. Run it locally or behind an authenticating proxy.